KUKU AI GENERALIST ASSIGNMENT — PROBLEM 1 ENGINEERING BLUEPRINT

Kuku Promo Finder — Lean Implementation Plan (v3.1)

Track: Find the Promos (Ads) Across Microdrama Series | **Revision:** v3.1 Final (Pre-Implementation Audit & Safeguards Integrated) | **Target Budget:** 150 Minutes (Planning Target)

Executive Summary: Final Pre-Implementation Baseline (v3.1)

This document defines the final, ready-to-build **v3.1 Lean Implementation Plan** for the Kuku Promo Finder prototype. It retains the four-module deterministic/LLM architecture, full screenplay element ingestion, integer centisecond timestamp math, and per-batch checkpoints, while incorporating **six final pre-implementation fixes**: (1) attaching full source AST screenplay elements to every narrative moment during cross-episode synthesis, (2) mechanically validating `0.0 <= hook_timestamp_offset_seconds <= 3.0`, (3) establishing a strict five-step rubric-sort-then-critic-review sequence that never marks unreviewed backfill candidates as critic-approved, (4) restricting duration extensions so cliffhangers are never accidentally resolved, (5) limiting deduplication to very high overlap (`>= 0.85` containment) or shared climax endpoints, and (6) standardizing the three promo archetypes while separating cross-episode stitching as a composition capability.

1. Technical Audit & Final Pre-Implementation Fixes

To build a working, convincing prototype across all 50 episodes of *"I Married a Broken Soldier. He was a Duke"* within the target 150-minute engineering budget, the architecture avoids unnecessary abstraction while protecting against runtime edge cases that would silently degrade recommendation quality.

1.1 Core Pre-Implementation Fixes & Zero-Bloat Safeguards

Identified Vulnerability	Why It Could Fail or Degrade Output Quality	Final v3.1 Implementation Rule
Fix 1: Synthesizer Context Starvation *(High Priority)*	Summaries, aliases, and durations alone are insufficient to verify whether two moments across episodes genuinely connect. For example, a ring discovered in Ep 7 and an identity reveal in Ep 42 might look connected in summaries while actual dialogue proves the ring belongs to someone else.	**Attach Verbatim Source Elements from AST:** Whenever Python prepares extracted `narrative_moments` for `synthesizer.py`, it looks up `start_element_id..end_element_id` in `01_structured_episodes.json` and attaches the **exact source screenplay elements** (speaker, action/camera tags, and verbatim dialogue) plus calculated duration directly to the synthesis prompt.

Identified Vulnerability	Why It Could Fail or Degrade Output Quality	Final v3.1 Implementation Rule
Fix 2: Unverified 3-Second Hook Timing *(High Priority)*	A candidate could receive a `hook_strength` score of 5 because of an explosive confrontation at second 12, even though the first 3 seconds are slow exposition—violating Kuku's 3-second hook requirement.	**Explicit Offset Field & Mechanical Gate:** Require every candidate to include `"hook_timestamp_offset_seconds"` (and verify it against the designated hook element offset in Python). Mechanically enforce `0.0 <= hook_timestamp_offset_seconds <= 3.0` during validation.
Fix 3: Ambiguous Critic & Ranking Order *(High Priority)*	Sorting by `critic_approved` before rubric scores when only the Top 15 are reviewed creates ambiguity: candidates ranked 16+ are unreviewed, and if the critic rejects a Top-15 clip, unreviewed clips could enter the final list appearing critic-approved.	**Strict 5-Step Ranking & Refill Pipeline:** (1) Mechanically validate & deduplicate all candidates; (2) Sort all valid candidates by the 4 rubric scores; (3) Send the initial Top 15 to the Critic; (4) Remove/demote explicitly rejected clips; (5) Refill from rank 16+ if needed, explicitly tagging any unreviewed backfill as `"critic_status": "UNREVIEWED_BACKFILL"`.
Fix 4: Duration Fallback Resolving Cliffhangers	Automatically appending `end_element_id + 1` to rescue a `17.00s–19.99s` clip can introduce trailing dialogue that answers the dramatic question and ruins the cliffhanger.	**Default to Rejection if Cliffhanger Risk Exists:** Format element durations in the prompt (targeting `25.00s–55.00s`). Only allow a `+1` element extension if it is in the same scene, stays `<= 60.00s`, and does not resolve the cliffhanger; if that cannot be verified cheaply, **reject invalid durations outright** rather than auto-extending.
Fix 5: Over-Aggressive Deduplication	Dropping any candidate with `intersection / min(dur_a, dur_b) >= 0.60` can eliminate a distinct promo that shares a transition beat while focusing on a different narrative climax.	**High-Overlap or Shared-Climax Rule:** Only deduplicate two same-episode candidates when overlap is very high (`intersection / min(dur_a, dur_b) >= 0.85`) **or** when they share the same `≥ 0.60` overlap *and* the exact same climax/ending element (`end_element_id`).
Fix 6: Archetype Naming Consistency	Mixing cross-episode stitching with single-episode dramatic structures in archetype lists can confuse extraction prompts.	**Standardized 3 Archetypes Everywhere:** Use (1) *Immediate Shock Opener*, (2) *Visual Anomaly / Mystery Escalation*, and (3) *Emotional Reversal / Turning Point* across all prompts and schemas. Treat cross-episode stitching strictly as a separate `composition_mode` (`single_episode` vs. `multi_episode`).

Identified Vulnerability	Why It Could Fail or Degrade Output Quality	Final v3.1 Implementation Rule
Fix 7: Batch Checkpoints & Prompt Ordering	A failed API call on batch 4 or output truncation wastes time, while placing episode context above clip text in the critic prompt causes "curse of knowledge" bias.	**Per-Batch Disk Checkpoints & Cold-First Critic Prompt:** Save `batch_1.json..batch_5.json` immediately after each call, and place the isolated clip transcript *before* surrounding context in the critic prompt.

1.2 Complete Scope Decision Matrix (All 22 Architectural Decisions)

1.2 Complete Scope Decision Matrix (Evaluating All 22 Proposed Adjustments)

Component	Decision	User Recommendation	Engineering Assessment & v3.1 Integration
Python screenplay parser	**Keep**	Simple deterministic parser	**Essential:** 100% deterministic state machine; zero LLM involvement in text parsing.
Integer centisecond timestamps	**Keep**	Very easy and useful	**Essential:** Converts `MM:SS.ss` to `int_cs (1s = 100 cs)`; eliminates float drift.
Full screenplay ingestion	**Keep**	Don't discard camera/action elements	**Essential:** All 50 episodes total ~98K tokens; keeps all 1,754 `CAMERA` and 372 `ACTION` blocks.
Tier A promotional candidates	**Keep**	Main candidate discovery	**Essential:** Primary single-episode promo span discovery in `extractor.py`.
Tier B1/B2 subregistries	**Simplify**	One list of important narrative moments, tagged by type	**Sound Simplification:** Unified `narrative_moments` list tagged as `setup`, `clue_or_prop`, `reveal`, `confrontation`, or `contrast_baseline`, stamped with Python duration.
NarrativeLedger	**Simplify**	Character/prop aliases and short episode context	**Sound Simplification:** Flat JSON context store (aliases, props/clues, key relationships, episode summaries) citing source element IDs.
Typed causal relationships	**Simplify**	Explicit, implied or thematic; no elaborate graph	**Sound Simplification:** Multi-segment stitches carry a simple `link_type ∈ {explicit, implied, thematic}` plus a 1-sentence justification.

Component	Decision	User Recommendation	Engineering Assessment & v3.1 Integration
Retroactive AST retrieval	**Simplify**	Basic keyword and alias lookup	**Sound Simplification:** Simple Python case-insensitive substring search over AST text and `CONTINUITY NOTES` for character/prop names.
22,500-window enumeration	**Remove**	Too much unnecessary computation	**High-Value Cut:** LLM directly identifies candidate start/end element IDs from batch transcripts; Python verifies boundaries.
Three promo archetypes	**Keep**	Include them as guidance in the extraction prompt	**Standardized:** Consistent across all prompts: (1) *Immediate Shock Opener*, (2) *Visual Anomaly / Mystery Escalation*, and (3) *Emotional Reversal / Turning Point*. Cross-episode stitching is a separate composition capability.
Boundary snapper	**Remove**	Reject invalid durations rather than trying to repair them	**Strict Rule:** Print element durations in prompt (`target 25s–55s`) and reject invalid durations (`< 20.00s` or `> 60.00s`) unless a 1-element same-scene extension is verified not to resolve the cliffhanger.
Exact timestamp validation	**Keep**	Essential	**Essential:** Python verifies `start_element_id` and `end_element_id` against immutable AST timestamps plus `0 <= hook_timestamp_offset_seconds <= 3`.
Three-regime validation	**Simplify**	Mechanical checks + one LLM quality review	**Sound Simplification:** Combines hard Python mechanical checks with a single Top-15 LLM Quality Critic review pass.
Pareto ranking	**Remove**	Unnecessary complexity	**High-Value Cut:** Replaced by deterministic rubric ordering followed by Top-15 Critic review and explicit backfill labeling.
Harmonic funnel index	**Remove**	Unvalidated mathematical sophistication	**High-Value Cut:** Eliminates arbitrary non-linear weighting over subjective 1–5 integer scores.
Archetype eligibility profiles	**Remove**	Use one consistent quality rubric	**High-Value Cut:** All candidates evaluated on the same 4-criterion rubric.

Component	Decision	User Recommendation	Engineering Assessment & v3.1 Integration
Dual-view critic	**Simplify**	One reviewer prompt with both perspectives	**Sound with Guardrail:** Single LLM review on the rubric-sorted Top 15 candidates containing isolated clip text first and surrounding `±3` elements second.
NMS deduplication	**Simplify**	Basic overlap-based deduplication	**Refined Rule:** Deduplicate only when same-episode containment overlap is very high (`>= 0.85`) or when clips overlap (`>= 0.60`) and share the same climax/ending element.
Negative-control benchmark	**Remove**	Not necessary for the assignment	**High-Value Cut:** Moved to Future Improvements.
Manual reference audit	**Keep**	Inspect a small sample	**Essential:** Mandatory full 50-episode run + manual audit of 5–10 representative episodes and Top 10–15 final recommendations.
Three-run stability testing	**Remove**	Too time-consuming	**High-Value Cut:** Saves 2× API execution time during the 150-minute build window.
Production cloud infrastructure & Confidentiality	**Defer / Keep**	Defer cloud infra; keep basic strict confidentiality	**Essential:** Local CLI execution; strict `.gitignore` , env vars, explicit verification of the exact API provider/endpoint/account retention policy, private raw output bundle.

2. Empirical Input Baseline vs. Planning Assumptions

The v3.1 prototype operates on the complete, unaltered 50-episode screenplay corpus for *"I Married a Broken Soldier. He was a Duke"*. To maintain strict engineering honesty, verified measurements of the static input files are explicitly separated from runtime planning assumptions (which depend on model choice, output verbosity, retry behavior, and API latency):

Parameter	Value / Target	Classification	Engineering Notes & Contingency
Screenplay Corpus Volume	50 `.txt` files; 75,120 words; 15,958 lines (~98K raw tokens)	**Verified Fact**	Measured directly from input files; ~19,600 raw tokens per 10-episode batch.
Series Duration & Elements	6,048.00s (~100.8 min); 3,725 timed elements	**Verified Fact**	1,754 `CAMERA` (47.1%), 1,485 `DIALOGUE` (39.9%), 372 `ACTION` (10.0%), 114 `SOUND FX` (3.1%).

Parameter	Value / Target	Classification	Engineering Notes & Contingency
Timestamp Resolution	`MM:SS.ss` (centiseconds, `1s = 100 cs`)	**Verified Fact**	Hard duration bounds enforced in integer centiseconds: `2,000 cs <= duration_cs <= 6,000 cs`.
API Call Volume	7 baseline calls (5 batch + 1 synth + 1 critic)	**Planning Assumption**	Assumes zero retries or batch splits; if a 10-episode batch hits output token limits, split into 5-episode sub-batches (up to 12–15 calls).
Wall-Clock Execution Time	~2–4 minutes per full pipeline run	**Planning Assumption**	Depends on provider rate limits, model reasoning latency, and any retry backoffs.
First-Pass Duration Compliance	Empirical runtime outcome	**Planning Assumption**	Pre-computing element durations in the prompt reduces arithmetic errors, but exact compliance rate is verified during execution.
API Run Cost & Code Size	Model-dependent cost; ~350–450 LOC	**Planning Assumption**	Cost depends on selected model tier and pricing; LOC scales with error-handling and schema validation rigor.
Implementation Timeline	150 minutes (2 hr 30 min)	**Target Budget**	Target engineering allocation, not a guarantee that every edge case resolves on the first pass.

3. The v3.1 Lean Four-Module Architecture

The prototype retains the clean four-module pipeline structure, where each module reads and writes a static JSON checkpoint file to disk so any stage can be inspected or re-run independently.

```
Input: 50 Unchanged Screenplay Files (input_scripts/ep01.txt .. ep50.txt)
   │
   ▼
[1. parser.py] (Deterministic Python)
   ├─ Parses EPISODE LOGLINE, CONTINUITY NOTES, scenes, and all 4 element types
(DIALOGUE, ACTION, CAMERA, SOUND FX)
   ├─ Converts MM:SS.ss timestamps to integer centiseconds (start_cs, end_cs,
duration_cs)
   ├─ Assigns immutable element IDs (e.g., EP01_EL001 .. EP50_EL078)
   └─ Emits: 01_structured_episodes.json (3,725 elements across 50 episodes)
   │
   ▼
[2. extractor.py] (LLM-Assisted, 5 Batches of 10 Episodes)
   ├─ Formats episodes with pre-computed decimal durations per element to minimize
LLM timestamp arithmetic errors
   ├─ Uses Three Consistent Promo Archetypes as prompt guidance:
   │    1. Immediate Shock Opener  |  2. Visual Anomaly / Mystery Escalation  |  3.
Emotional Reversal / Turning Point
   ├─ Extracts single-episode Promo Candidates (start/end IDs,
hook_timestamp_offset_seconds, hook/cliffhanger/coherence, 1–5 scores)
   ├─ Extracts one unified list of tagged Narrative Moments + flat Narrative
Context (aliases, clues, relationships, summaries)
   ├─ Python stamps exact AST duration_seconds AND verbatim source screenplay
elements onto every extracted Narrative Moment
   └─ Emits: 02_batch_extractions.json (checkpointed per batch: batch_1.json ..
batch_5.json)
   │
   ▼
[3. synthesizer.py] (Python AST Attachment + 1 LLM Call)
   ├─ Passes episode summaries, alias table, keyword-matched AST context, AND
verbatim source screenplay elements for each moment
   ├─ Combines suitable moments across episodes (2–3 segments) as a cross-episode
synthesis capability (explicit, implied, thematic)
   ├─ Python validates multi-segment continuity, AST timestamp existence, 0 <=
hook_offset <= 3s, and total duration (20.00s–60.00s)
   └─ Emits: 03_synthesized_candidates.json
   │
   ▼
[4. validator_ranker.py] (Hybrid Python + 1 LLM Quality Critic Call)
   ├─ Step 1 (Mechanical Gate): Validates exact AST timestamps, 20.00s–60.00s
duration, and 0.0 <= hook_timestamp_offset_seconds <= 3.0
   ├─ Step 2 (High-Overlap Dedup): Removes duplicates only when containment overlap
>= 0.85 or shared climax endpoint (>= 0.60 overlap)
   ├─ Step 3 (Rubric Sort): Orders all valid candidates deterministically using the
4 rubric scores
   ├─ Step 4 (Top-15 Quality Critic): Reviews initial Top 15 against the 5
editorial questions; removes/demotes rejected candidates
   ├─ Step 5 (Explicit Backfill): Refills Top 15 from rank 16+ if needed, marking
any unreviewed backfill as UNREVIEWED_BACKFILL
   └─ Emits: final_promos.json, raw_llm_outputs/, editorial_report.md,
execution_trace.json
```

3.1 Module-by-Module Implementation Contract

Module	Mode	Core Responsibilities	Key Output Artifacts
1. `parser.py`	Deterministic Python	Parse episode headers (`EPISODE LOGLINE` , `CONTINUITY NOTES`), scene boundaries, and timed elements (`DIALOGUE` , `ACTION` , `CAMERA` , `SOUND FX`). Convert `MM:SS.ss` to `int_cs` , verify `0 <= start_cs < end_cs` , and assign sequential IDs (`EP07_EL014`).	`01_structured_episodes.json` (50 episodes, 3,725 elements)
2. `extractor.py`	LLM-Assisted (5 batches)	Process 5 batches of 10 episodes. Identify single-episode candidates (guided by the 3 standard archetypes), record `hook_timestamp_offset_seconds` , extract one list of tagged narrative moments, and build episode summaries + character/prop aliases. Python attaches exact AST durations and verbatim source elements to every moment.	`02_batch_extractions.json` & per-batch checkpoints in `raw_llm_outputs/`
3. `synthesizer.py`	Python + LLM	Retrieve and attach the exact source screenplay elements from the AST for every candidate narrative moment (plus any keyword/alias lookups). Prompt the LLM with the actual scene scripts, summaries, and aliases to stitch 2–3 segments across episodes. Check narrative continuity and validate total duration (20.00s–60.00s).	`03_synthesized_candidates.json`
4. `validator_ranker.py`	Hybrid Python + LLM	Validate source timestamps, 20.00s–60.00s duration, and `0 <= hook_timestamp_offset_seconds <= 3.0` . Deduplicate high-overlap/same-climax spans, sort by the 4 rubric scores, run one LLM Quality Critic pass on the Top 15, demote/remove rejected clips, refill from next highest-scoring candidates if needed, and emit final deliverables.	`final_promos.json` , `editorial_report.md` , `execution_trace.json`

4. Detailed Specification of the Seven Architectural Revisions in v3.1

Revision 1: Flat Context Store, Tagged Moment List & Verbatim AST Attachment for Synthesis

Instead of building an entity-resolution graph, `extractor.py` populates two lightweight JSON structures per 10-episode batch that Python merges across all 5 batches and enriches directly from the AST:

- **Flat `narrative_context` Store:** Stores (1) character names and surface aliases (e.g., `"Alexander" <-> ["The Broken Soldier", "The Crippled Veteran", "Duke of Ashburn"]`), (2) important objects and clues (signet ring, marriage contract, poison vial) with episode/element IDs, (3) key character relationships, and (4) 1–2 sentence episode summaries.
- **Unified `narrative_moments` List with Attached Source Elements:** Replaces separate Tier B1/B2 subregistries with one flat list of cross-episode building blocks, each tagged with `moment_type` (`setup_or_vow`, `clue_or_prop`, `identity_reveal`, `high_stakes_confrontation`, or `contrast_baseline`), `start_element_id`, `end_element_id`, a brief summary, and optional `lookup_keywords`. **Crucially (solving Problem 1), before `synthesizer.py` calls the LLM, Python retrieves the exact screenplay elements (`start_element_id..end_element_id`) from `01_structured_episodes.json` and attaches both `duration_seconds` and the verbatim dialogue/action/camera text to every moment.** This ensures the synthesizer evaluates the actual dialogue—never just summaries—before stitching moments across episodes.

```
{
  "narrative_context": {
    "character_aliases": [
      {"canonical_name": "Alexander (Duke of Ashburn)", "aliases": ["The Broken
Soldier", "Crippled Veteran"], "key_episodes": [1, 7, 42]},
      {"canonical_name": "Juliana", "aliases": ["The Abandoned Bride", "Duchess of
Ashburn"], "key_episodes": [1, 2, 7, 42]}
    ],
    "objects_and_clues": [
      {"item": "Ashburn Ducal Signet Ring", "element_id": "EP07_EL038",
"significance": "Proves the scarred soldier is the presumed-dead Duke"}
    ],
    "relationships": [
      {"pair": "Juliana & Gerald", "relation": "Former fiancé who publicly
abandoned her at the altar in Ep 1 (EP01_EL012)"}
    ]
  },
  "narrative_moments": [
    {
      "moment_id": "EP01_MOM_01",
      "episode": 1,
      "moment_type": "contrast_baseline",
      "start_element_id": "EP01_EL012",
      "end_element_id": "EP01_EL019",
      "duration_seconds": 17.40,
      "summary": "Gerald publicly humiliates and abandons Juliana at the altar.",
      "lookup_keywords": ["altar", "Gerald", "abandoned"],
      "source_elements_verbatim": [
        "[EP01_EL012 | 00:24.10-00:28.50] **GERALD (COLDLY):** \"I cannot wed a
woman whose family name is ruined.\"",
        "[EP01_EL013 | 00:28.50-00:31.00] **ACTION:** Gerald drops Juliana's hand
at the altar steps as the congregation gasps."
      ]
    }
  ]
}
```

Revision 2: Direct LLM Candidate Identification & Standardized Three Archetypes

Combinatorial enumeration of 22,500 element pairs is eliminated. Instead, `extractor.py` feeds the complete 10-episode batch transcript to the LLM with every element formatted with its ID, timestamp, and pre-calculated duration: `[EP01_EL012 | 00:24.10 -> 00:28.50 | dur=4.40s] **GERALD (COLDLY):** "..."`. Across the entire system, all prompts and schemas use the exact same **Three Promo Archetypes**:

1. **Immediate Shock Opener (`immediate_shock_opener`):** Opens within <= 3.00s on a visceral physical action, slap, betrayal accusation, or high-stakes threat and cuts on an unresolved cliffhanger.
2. **Visual Anomaly / Mystery Escalation (`visual_anomaly_mystery_escalation`):** Opens within <= 3.00s on a suspicious visual beat in a `CAMERA` or `ACTION` block (e.g., poison vial, royal seal, hidden scar) that rapidly escalates to a confrontation twist.
3. **Emotional Reversal / Turning Point (`emotional_reversal_turning_point`):** Opens within <= 3.00s on a sharp status humiliation, vow, or power inversion that builds to an unresolved emotional climax.

Cross-Episode Stitching as a Separate Capability: Cross-episode synthesis in `synthesizer.py` is treated as a separate candidate-generation mode (`composition_mode: "multi_episode"` vs. `"single_episode"`), not a fourth archetype. Every candidate (whether single-episode or multi-episode) is tagged with which of the three archetypes best describes its dramatic structure, along with `start_element_id`, `end_element_id`, `hook_timestamp_offset_seconds`, `hook_description`, `cliffhanger_description`, `standalone_coherence_explanation`, optional `trim_quote_hint`, and the four 1–5 rubric scores.

Revision 3: Simple Four-Criterion Rubric & Unambiguous Five-Step Ranking Pipeline

Pareto frontier stratification, harmonic funnel indices, archetype eligibility profiles, and pairwise arbitration are removed. Every candidate is scored on one consistent 1–5 integer rubric:

Criterion	Score	Operational Scoring Guide
Standalone Coherence	1–5	**5:** Conflict, stakes, and character dynamic are 100% clear to a first-time viewer with zero prior episodes. **4:** Clear conflict with an intentional curiosity gap. **3:** Mostly clear but fast-paced. **1–2:** Confusing ungrounded pronouns or missing backstory.
Hook Strength	1–5	**5:** Immediate visceral shock, action, or explosive line within 0–2s. **4:** Strong dramatic tension or visual anomaly within <= 3s. **3:** Moderate setup within 3s. **1–2:** Slow atmospheric opening (> 3.0s).
Cliffhanger Strength	1–5	**5:** Cuts at peak unresolved jeopardy, ultimatum, or bombshell reveal before characters finish reacting. **4:** Strong open dramatic question. **3:** Mild tension. **1–2:** Scene resolves its conflict before the cut.
Emotional Impact	1–5	**5:** High-intensity betrayal, moral outrage, romantic peril, or cathartic vindication. **3:** Standard dramatic exchange. **1:** Flat exposition.

Unambiguous Five-Step Validation, Ranking & Critic Refill Sequence (Solving Problem 3): To ensure unreviewed candidates are never conflated with critic-approved candidates, `validator_ranker.py` executes five sequential steps:

1. **Step 1 — Mechanical Validation & Deduplication:** Validate exact AST element timestamps, total duration (`20.00s <= duration <= 60.00s`), and hook timing (`0.0 <= hook_timestamp_offset_seconds <= 3.0`) across all single- and multi-episode candidates. Deduplicate overlapping spans within the same episode only when containment overlap is very high (`intersection_cs / min(dur_a_cs, dur_b_cs) >= 0.85`) or when overlap is `>= 0.60` and both clips share the exact same climax/ending element (`end_element_id`).
2. **Step 2 — Deterministic Rubric Sort:** Sort all mechanically valid, deduplicated candidates purely by their rubric scores (prioritizing standalone coherence and hook strength, followed by cliffhanger strength and emotional impact):

```
rubric_sort_key = (
    passes_min_floor,         # Boolean: standalone >= 4 and hook >= 3 and
cliffhanger >= 3
    standalone_coherence,     # Primary priority (1..5)
    hook_strength,            # Secondary priority (1..5)
    cliffhanger_strength,     # Tertiary priority (1..5)
    emotional_impact,         # Quaternary priority (1..5)
    total_rubric_score        # Sum of the 4 scores (4..20)
)
```

3. **Step 3 — Top-15 Quality Critic Review:** Send the initial Top 15 rubric-sorted candidates to the single LLM Quality Critic call.
4. **Step 4 — Demote or Remove Rejected Candidates:** Any candidate in the initial Top 15 that receives a `REJECT` or `DEMOTE` verdict from the critic (e.g., for accidental cliffhanger resolution, ungrounded pronouns, or incoherent stitching) is removed from the top recommendation tier and logged with the critic's objection.
5. **Step 5 — Refill from Next Highest-Scoring Candidates:** If demotions reduce the top list below 15, refill the remaining slots in deterministic rubric order from candidates ranked 16 onwards (or send a 20-candidate buffer in Step 3 so replacements are also reviewed). Any candidate entering the final list without critic review is explicitly labeled `"critic_status": "UNREVIEWED_BACKFILL"` —never `"APPROVE"`.

Revision 4: Strict Duration Validation, 3-Second Hook Gate & Safe Boundary Handling

Complex boundary snappers and character-count linear timestamp interpolations are removed:

- **Exact AST Timestamp & 3-Second Hook Verification (Solving Problem 2):** Python looks up `start_cs = AST[start_element_id].start_cs` and `end_cs = AST[end_element_id].end_cs`, and verifies both `2,000 cs <= duration_cs <= 6,000 cs` and `0.0 <= hook_timestamp_offset_seconds <= 3.0` (additionally checking that the designated opening hook element initiates within `300 cs` of `start_cs`).

- **Safe Handling of Out-of-Bounds Durations (Solving Problem 4):** Because element durations are printed directly in the extraction prompt and the LLM is instructed to target `25.00s-55.00s`, candidates generally land within bounds. If a candidate has an invalid duration (`< 20.00s` or `> 60.00s`), the default action is to **reject it and log the rejection in `execution_trace.json`**. A `17.00s-19.99s` candidate may only be extended by `+1` adjacent element if (a) the element belongs to the same scene, (b) total duration stays `<= 60.00s`, and (c) the extension is verified not to resolve the original cliffhanger; if condition (c) cannot be verified cheaply, reject the candidate rather than extending it automatically.
- **Verbatim Quote Hints for Internal Sentence Cuts:** For internal sentence-level editing inside a long element, the LLM returns the relevant verbatim quote as a hint (`trim_quote_hint`). Python never invents interpolated centisecond timestamps using character counts.

Revision 5: Simplified Single-Call Quality Critic on the Top 15 Candidates

In Step 3 of `validator_ranker.py`, one LLM call reviews the Top 15 rubric-ranked candidates. For each candidate, the prompt presents (a) the isolated clip transcript first, followed by (b) the `±3` surrounding screenplay elements and episode logline. The critic evaluates five explicit questions:

1. **Cold Comprehension:** Does the clip make sense without previous episodes (distinguishing intentional curiosity gaps from ungrounded pronoun confusion)?
2. **3-Second Opening:** Is the opening compelling within the first three seconds (matching `hook_timestamp_offset_seconds <= 3.0`)?
3. **Unresolved Ending:** Does the ending leave an unresolved dramatic question or cliffhanger?
4. **Spoiler / Over-Reveal Guard:** Does the clip accidentally reveal too much (resolving the conflict before the cut)?
5. **Multi-Episode Continuity:** Does a multi-episode combination preserve genuine narrative meaning across the actual source dialogue?

The critic returns explicit objections, editorial recommendations, and a `critic_status` of `"APPROVE"`, `"FLAG_WITH_OBJECTION"`, or `"DEMOTE"`.

Revision 6: Practical Human Audit on the Mandatory Full 50-Episode Run

Multi-run stability testing, multi-annotator benchmarks, and synthetic negative controls are removed in favor of a practical engineering audit:

- **Mandatory Full Dataset Execution:** Process all 50 episodes end-to-end and save all intermediate and final artifacts.
- **Representative Episode Inspection (5–10 Episodes):** Manually inspect extracted outputs from 5–10 representative episodes across early setups, mid-series twists, and late climaxes (e.g., Episodes 1, 4, 7, 15, 24, 33, 42, 50) to verify whether the best moments were discovered.
- **AST Timestamp & Hook Offset Verification:** Verify 100% of recommended timestamps and hook offsets against `01_structured_episodes.json` and raw screenplay files.

- **Top 10–15 Inspection & Honest Failure Documentation:** Inspect the top 10–15 final recommendations in detail and document any missed beats, rejected durations, or weak stitches honestly in `editorial_report.md`.

Revision 7: Simple, Strict Confidentiality Safeguards (With Explicit Policy Verification)

Enterprise cloud infrastructure (VPCs, Cloud Run, IAM) is deferred. Confidentiality for the prototype is enforced through five strict rules:

1. Keep all 50 screenplay `.txt` files and raw output logs out of public GitHub via a strict `.gitignore`.
2. Keep API keys in local environment variables (`os.environ["GEMINI_API_KEY"]`); never hardcode credentials in source files.
3. **Verify Exact API Provider Data Policies (No Blind Paid-Tier Assumptions):** Do not assume that a paid API tier automatically guarantees a specific data-retention or training policy. Explicitly verify the selected provider, endpoint, and account terms before uploading confidential screenplays.
4. Keep all raw LLM outputs, full JSON checkpoints, and the editorial report inside the private submission bundle.
5. Never claim "zero data retention" or "zero leakage" without verified contractual documentation.

5. Concrete Output Schema (`final_promos.json`)

Every ranked promo emitted by `validator_ranker.py` conforms to the following schema, including `hook_timestamp_offset_seconds`, standardized archetype naming, and explicit critic review status:

```
{
  "promo_id": "PROMO_001",
  "rank": 1,
  "composition_mode": "multi_episode",
  "archetype": "emotional_reversal_turning_point",
  "relationship_type": "thematic",
  "total_duration_seconds": 38.40,
  "total_duration_cs": 3840,
  "hook_timestamp_offset_seconds": 1.20,
  "segments": [
    {
      "segment_index": 1,
      "episode": 1,
      "scene": 2,
      "start_element_id": "EP01_EL012",
      "end_element_id": "EP01_EL019",
      "start_timestamp": "00:24.10",
      "end_timestamp": "00:41.50",
      "duration_seconds": 17.40,
      "trim_quote_hint": null,
      "segment_summary": "Groom Gerald publicly humiliates and abandons Juliana at
the altar."
    },
    {
      "segment_index": 2,
      "episode": 42,
      "scene": 1,
      "start_element_id": "EP42_EL008",
      "end_element_id": "EP42_EL017",
      "start_timestamp": "00:18.00",
      "end_timestamp": "00:39.00",
      "duration_seconds": 21.00,
      "trim_quote_hint": "Cut immediately after: 'On your knees before the Duchess
of Ashburn.'",
      "segment_summary": "The scarred soldier reveals his identity as Duke of
Ashburn and forces Gerald to kneel."
    }
  ],
  "rubric_scores": {
    "standalone_coherence": 5,
    "hook_strength": 5,
    "cliffhanger_strength": 5,
    "emotional_impact": 5,
    "total_score": 20
  },
  "editorial_rationale": {
    "hook_explanation": "Opens at 1.20s on a visceral public altar rejection ('I
cannot wed a woman whose family name is ruined').",
    "cliffhanger_explanation": "Cuts at peak status inversion as the Duke orders
the former groom to his knees.",
    "standalone_coherence_explanation": "The humiliation-to-vindication reversal is
immediately clear to a cold viewer without intermediate episodes."
  },
  "critic_review": {
    "critic_status": "APPROVE",
    "makes_sense_standalone": true,
    "compelling_within_3s": true
```

6. Revised 150-Minute Implementation Timeline & Stop-Loss Rules

This 150-minute (2 hr 30 min) schedule treats the **25-minute full execution and output inspection block as sacred**. If any coding phase approaches its time cap, simplify the prompt or formatting rather than borrowing time from full-dataset execution and manual inspection.

Phase	Implementation Task	Allocation	Concrete Deliverable & Stop-Loss Guardrail
1	Environment & input inspection	**10 min**	Set up project directory, `.gitignore`, API key env var, and inspect raw screenplay formatting across 3 sample files.
2	Parser & timestamp validation (`parser.py`)	**25 min**	Parse all 50 files into `01_structured_episodes.json`; assert 50 episodes, 3,725 elements, and valid `int_cs` timestamps.
3	Candidate extraction across 50 episodes (`extractor.py`)	**35 min**	Implement 5×10 batch prompt with 3 archetypes, tagged moments, and flat context store; checkpoint `batch_1..5.json` and attach AST durations.
4	Cross-episode synthesis (`synthesizer.py`)	**20 min**	Run basic keyword/alias AST lookup and 1 LLM call to combine narrative moments across episodes; check continuity and 20s–60s total duration.
5	Ranking & quality validation (`validator_ranker.py`)	**20 min**	Run mechanical timestamp/duration gates, containment overlap deduplication (≥0.60), deterministic rubric sort, and 1 LLM Critic call on Top 15.
6	**Full execution & output inspection**	**25 min**	**Mandatory:** Verify full 50-episode output, audit 8 representative episodes + Top 15 recommendations against AST, and document failure cases honestly.
7	Submission packaging	**15 min**	Generate `editorial_report.md`, organize `raw_llm_outputs/` and `execution_trace.json`, and package private deliverables.
Total Prototype Budget		**150 min**	**2 Hours 30 Minutes — End-to-End Working Prototype & Verified Outputs**

7. Future Improvements (Explicitly Removed from v3.1 Prototype Scope)

To keep the prototype focused, transparent, and buildable within 2.5 hours, the following ten components from earlier v3 drafts are explicitly excluded from implementation and archived as post-prototype extensions:

1. **22,500-Window Combinatorial Enumeration:** Unnecessary when the LLM reads complete screenplay batches with element IDs and pre-computed durations.
2. **Full Entity-State & Provenance Graph (`NarrativeLedger` Graph):** Replaced by a flat context store (character aliases, key clues/props, relationships, and short episode summaries) plus one tagged list of narrative moments.
3. **Harmonic Funnel Formula:** Replaced by transparent 1–5 integer rubric scores.
4. **Pareto Frontier Calculations:** Replaced by deterministic lexicographic ordering with a minimum rubric floor.
5. **Position-Swapped Pairwise Arbitration:** Eliminated in favor of a single Top-15 Quality Critic review pass.
6. **Three-Run Fuzzy Stability Benchmarking:** Replaced by single-run deterministic execution logging and manual inspection.
7. **Ten-Episode Multi-Annotator Research Benchmark:** Replaced by a practical human audit of 8 representative episodes and the Top 10–15 recommendations.
8. **Synthetic Negative-Control Benchmark:** Unnecessary for demonstrating promo discovery on the target dataset.
9. **Enterprise Cloud Infrastructure (VPC, Cloud Run, IAM):** Deferred to post-prototype deployment; local CLI execution with strict environment and repository hygiene is sufficient.
10. **Parametric Annual TCO & Crossover Labor Trials:** Deferred to production rollout planning.